04 — Matrices (2D Grids) and Simulation

This is the heart of **Q3**: a long, detailed description you must implement *exactly*. No clever algorithm needed — just careful reading and clean 2D code. Q3 is won by being **careful**, not smart. Most people fail it by misreading a rule, not by lacking skill.

---

1. A matrix is a list of lists

A **matrix** (or grid) is just a **list whose items are themselves lists** — each inner list is one row.

```
grid = [
    [1, 2, 3],      # row 0
    [4, 5, 6],      # row 1
]
rows = len(grid)         # 2  — number of rows = length of the outer
cols = len(grid[0])      # 3  — number of columns = length of one ro
grid[1][2]               # 6  — grid[ROW][COLUMN]. Row FIRST, then c
```

> **The golden rule: `grid[r][c]` is ROW first, COLUMN second.** Swapping them (`grid[c][r]`) is the most common 2D bug. Say "row, column" out loud every time.

Visit every cell with a loop inside a loop — outer picks the row, inner picks the column:

```
for r in range(rows):
    for c in range(cols):
        do_something(grid[r][c])
```

> **Building a grid — the #1 trap:**
>
> ```
> grid = [[0] * cols for _ in range(rows)]   # ✅ CORRECT — each
> grid = [[0] * cols] * rows                 # ❌ BUG — all rows
> ```
>
> With the buggy version, changing `grid[0][0]` also changes `grid[1][0]`, `grid[2][0]`... because every row is secretly the same list. Always use the first (comprehension) form.

---

2. The neighbors pattern — MEMORIZE

Many Q3 problems ask about the cells *around* a position (Minesweeper, flood fill, counting). The challenge: don't fall off the edge of the grid. The pattern handles both.

Idea: list the *direction offsets* (how row and column change to reach each neighbor), then loop over them, checking bounds before you touch a cell.

```
# 4 directions (up, down, left, right):
directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

# 8 directions (add the 4 diagonals):
directions = [(-1,-1), (-1,0), (-1,1),
              ( 0,-1),         ( 0,1),
              ( 1,-1), ( 1,0), ( 1,1)]

for dr, dc in directions:
    nr, nc = r + dr, c + dc                  # neighbor's row, neigl
    if 0 <= nr < rows and 0 <= nc < cols:    # BOUNDS CHECK FIRST —
        ...use grid[nr][nc]...
```

Read `(-1, 0)` as "one row up, same column." The bounds check `0 <= nr < rows` means "row number is between 0 and the last row" — memorize it as one line. **Always check bounds BEFORE reading `grid[nr][nc]`**, or you'll crash at the edges.

Minesweeper (REAL GCA Q3): for each cell, count how many of its 8 neighbors are mines — that's exactly the loop above with a counter.

---

3. Rows, columns, diagonals

```
row_i = grid[i]                                    # a whole row — 
col_j = [grid[r][j] for r in range(rows)]          # a whole column
main_diagonal = [grid[i][i] for i in range(rows)]        # top-left
anti_diagonal = [grid[i][rows-1-i] for i in range(rows)] # top-righ
```

A **row** is easy — it's a single item, `grid[i]`. A **column** is spread across every row, so you gather it: "take position `j` from each row." That column comprehension is worth memorizing.

```
# Rotate 90° clockwise (GCA/interview classic):
rotated = [list(row) for row in zip(*grid[::-1])]
# Transpose (flip rows and columns):
transposed = [list(row) for row in zip(*grid)]
```

Don't memorize the `zip(*...)` lines blindly — but DO memorize the column comprehension above.

---

4. Simulation problems: "follow the rules"

Q3 often describes a step-by-step process. Your job: translate each sentence into code, **in order**, without being clever.

Method that works every time:

1. Read the whole statement **twice**.
2. Write each rule as a **comment** in your code first.
3. Fill in code under each comment.
4. **Trace the given example by hand** and compare with your code's output.

Worked example — matrixElementsSum (REAL GCA): *"A room's price is in the grid. A room is unavailable (haunted) if it's `0`, and every room directly **below** a haunted room is also unavailable. Sum the prices of available rooms."*

Reading it: walk each **column top to bottom**; the moment you hit a `0`, everything below in that column is blocked, so stop that column.

```
def matrix_elements_sum(matrix):
    total = 0
    for c in range(len(matrix[0])):       # go column by column
        for r in range(len(matrix)):      # walk that column top → l
            if matrix[r][c] == 0:
                break                      # hit a 0 → skip the rest
            total += matrix[r][c]          # otherwise the room is a
    return total
```

Trace `[[0,1],[1,2]]`: column 0 → top is 0 → break (add nothing). Column 1 → top is 1 (add 1), next is 2 (add 2). Total = 3. The `break` is the "everything below is blocked" rule in code.

Other Q3 shapes — `addBorder`, spiral traversal, "is this Sudoku row valid?" — are all careful loops, not algorithms. Read carefully, trace by hand.

---

5. Q3 checklist (this is where passes are won)

- ☐ Did I read the statement **twice**?
- ☐ Did I handle the **edges** (first/last row and column)?
- ☐ Rows vs columns — am I sure it's `grid[r][c]` and not `grid[c][r]`?
- ☐ What if the grid is **1×1**, or a single row, or a single column?

- [ ] Did I **trace the given example by hand** and match the expected output?
- [ ] Off-by-one: does my loop reach the last index? (`range(n)` stops at n−1.)

Exercises (in `practice/q3_implementation.py`)

1. `matrix_elements_sum(matrix)` — as above.
2. `minesweeper(matrix)` — replace each cell with the count of neighboring `True`s (8 directions).
3. `rotate_image(grid)` — rotate a square grid 90° clockwise WITHOUT `zip` (loops only, to understand it).
4. `are_similar(a, b)` — can list a become b by swapping at most one pair? (real GCA)
5. `spiral_numbers(n)` — fill an n×n grid with 1..n^2 in a clockwise spiral. (tough Q3-level — great practice)

Which idea for each? 1 → column-walk with `break` (above). 2 → the 8-direction neighbors loop with a counter per cell. 3 → new cell `[r][c]` comes from old `[rows-1-c][r]` — trace a 2×2 by hand first. 4 → collect the positions where `a` and `b` differ; it's "similar" if there are 0 such positions, or exactly 2 that are a mirror swap. 5 → keep a direction and turn when you'd hit an edge or a filled cell.